1. Executive Summary

Project Name: BGalarza CloudFolio — AWS Portfolio Website

Tagline: A scalable, cloud-native portfolio website deployed entirely on AWS infrastructure.

Problem Statement & Target Use Case

The goal of this project was to create and launch a complete portfolio website on AWS. The target use case is a college student with an IT background pursuing a cloud computing degree, providing a professional portfolio to show finished AWS projects and technical skills, along with a working contact form, all supported by reliable, flexible AWS infrastructure.

Tech Stack Overview

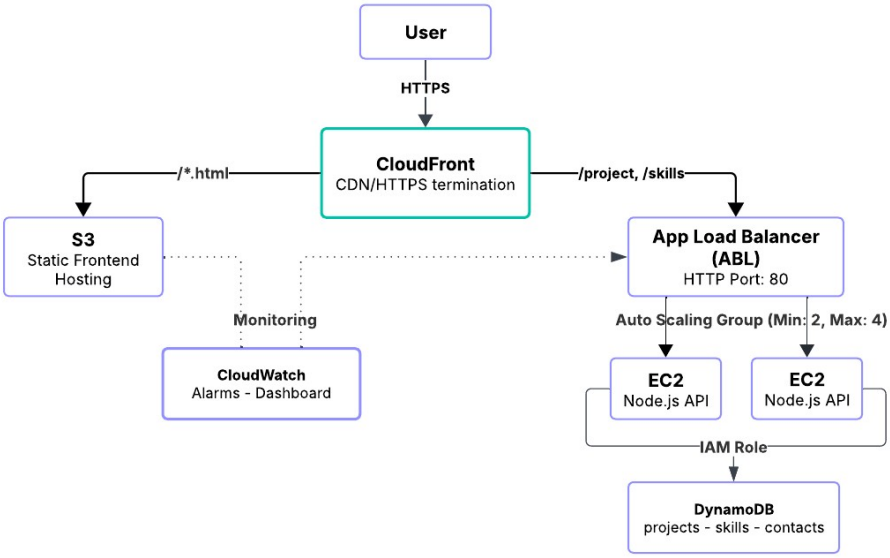
- Amazon EC2 — Application servers running Node.js/Express API
- Application Load Balancer (ALB) — Distributes traffic across EC2 instances
- Auto Scaling Group — Automatically maintains desired instance count
- Amazon DynamoDB — NoSQL database storing projects, skills, and contact submissions
- Amazon S3 — Hosts the static frontend (HTML/CSS/JS)
- Amazon CloudFront — CDN delivering the frontend globally over HTTPS
- AWS IAM — Role-based access control for EC2 instances
- Amazon CloudWatch — Monitoring dashboards and alarms

2. Architecture Design

Architecture Overview

This system uses a two-part structure. The frontend is a static website that is delivered to users through S3 and CloudFront, while the backend is an API that runs on EC2 servers behind a load balancer. All of the information is kept in DynamoDB.

Architecture Diagram



COMPONENT ROLES

VPC & Networking: A custom VPC (10.0.0.0/16) with public subnets across us-east-1a and us-east-1b, Internet Gateway, and route tables for public internet access.

Security Groups: The ALB accepts HTTP (port 80) from the internet; EC2 instances only accept port 3000 from the ALB security group, so no direct public access.

IAM Role: EC2 instances assume a role with DynamoDB read/write permissions without storing login credentials.

ALB + ASG: Application Load Balancer distributes traffic across EC2 instances. Auto Scaling Group maintains min: 2, max: 4 instances with automatic replacement of unhealthy instances.

DynamoDB: Three on-demand tables hold projects (6 items), skills (15 items), and contacts (form submissions). This was chosen instead of RDS because it is serverless and can easily scale.

S3 + CloudFront: The static frontend is stored in S3, with CloudFront used as a content delivery network. CloudFront directs requests for /projects, /skills, /health, and /contact to the load balancer, fixing HTTPS content problems.

JUSTIFICATION

- EC2 over Lambda: It provides a persistent Node.js server and full control over the runtime environment and configuration.
- DynamoDB over RDS: Its serverless, scalable NoSQL architecture fits a simple key-value data model with no relational joins.
- CloudFront over direct S3 access: It provides HTTPS support, a global content delivery network, and helps resolve mixed content issues when making API calls.
- ALB over a single EC2 instance: It distributes incoming traffic, performs health checks, and integrates smoothly with Auto Scaling.

3. IMPLEMENTATION DETAILS

SETUP INSTRUCTIONS

1. Create a VPC with CIDR 10.0.0.0/16, two public subnets in us-east-1a and us-east-1b, an Internet Gateway, and route tables.
2. Create Security Groups: one for the ALB (port 80 from 0.0.0.0/0) and one for EC2 (port 3000 from the ALB security group only).
3. Create an IAM Role with a DynamoDB full access policy and attach it as an instance profile.
4. Create three DynamoDB tables (projects, skills, contacts) and seed with initial data.
5. Create a Launch Template with Amazon Linux 2023 AMI, t2.micro, security group, IAM role, and user data script installing Node.js and starting the Express API.
6. Create Target Group (port 3000, HTTP, /health check) and Application Load Balancer across both subnets.
7. Create an Auto Scaling Group using the Launch Template, desired: 2, min: 2, max: 4, attached to the Target Group.
8. Create an S3 bucket with static website hosting enabled and a public read bucket policy.
9. Create CloudFront distribution with S3 as the default origin, ALB as secondary origin, and path behaviors for /projects, /skills, /health, and /contact.
10. Upload index.html to S3 and set index.html as the CloudFront default root object.

KEY CONFIGURATIONS

- CloudFront behaviors are configured with caching disabled and use the AllViewer origin request policy for all API routes. This setup ensures that users always receive the most up-to-date data.
- The ALB is set as an HTTP-only origin in CloudFront because it does not have an SSL certificate. This ensures that requests are sent using the correct protocol.
- For static website hosting, the S3 bucket uses the website endpoint as the origin in CloudFront instead of the REST endpoint. This allows the website to function properly and serve content as intended.
- The ASG is set to use the latest version of the Launch Template. This ensures that the most recent configuration is always applied when new instances are launched.

CHALLENGES & SOLUTIONS

Challenge: The Launch Template user data script created an empty index.js file on the EC2 instances.

Solution: Manually added the correct application code to /app/index.js on both instances using EC2 Instance Connect, then created a Launch Template v3 with a fixed user data script.

Challenge: Browser errors happened because CloudFront (using HTTPS) tried to connect to the ALB (using HTTP).

Solution: Added the ALB as a second CloudFront origin and created path-based behaviors for API routes, so all API calls route through CloudFront instead of directly to the ALB.

Challenge: CloudFront showed a 504 Gateway Timeout on the root URL after setting the default homepage.

Solution: Changed the S3 origin protocol from HTTPS to HTTP only, since S3 static website endpoints only support HTTP.

TESTING APPROACH

- Verified EC2 instances by testing http://localhost:3000, /health, /projects, and /skills directly on each instance to confirm the API was responding correctly
- Tested the Load Balancer DNS endpoint in a browser to confirm routing and DynamoDB data retrieval.
- Checked CloudFront delivery by visiting its domain and testing all four API routes.
- Verified Auto Scaling by terminating both original instances and confirming the ASG launched two new healthy instances.
- Submitted the contact form and checked that the data was saved in the DynamoDB contacts table

4. CLOUD ENGINEERING BEST PRACTICES

SECURITY

- IAM Role with minimal DynamoDB permissions attached to EC2 instances and no fixed passwords in the code.
- Security Groups follow the least privilege rule: EC2 instances only allow traffic from the ALB security group on port 3000, not from the public internet.
- S3 bucket access is controlled by a policy that only allows public reading of the website content.

- CORS is enabled on the Express API to control which sources can make requests.

SCALABILITY

- An Auto Scaling Group set to have 2 instances (minimum 2, maximum 4) automatically adds more instances when CPU usage exceeds 70%.
- DynamoDB on-demand capacity mode automatically scales to handle any traffic load without manual setup.
- CloudFront CDN caches static assets at edge locations globally, reducing the load on the main server.

HIGH AVAILABILITY

- EC2 instances and subnets are distributed across two Availability Zones (us-east-1a and us-east-1b).
- ALB covers both Availability Zones and automatically routes traffic only to healthy instances.
- If an instance fails, the ASG automatically launches a replacement to keep the desired count of 2.

COST ANALYSIS

The estimated monthly cost to run the BGalarza CloudFolio setup is about $33.88. The highest costs are the Application Load Balancer (~$16.43) and EC2 servers (~$16.06), which run continuously. DynamoDB, S3, CloudFront, and CloudWatch together cost less than $1.50 per month due to low usage and small storage. If you qualify for the AWS Free Tier, EC2 costs are covered for the first 12 months, lowering the monthly cost to about $17.82.

MONITORING

- CloudWatch Dashboard (bgalarza-cloudfolio-dashboard) with 4 widgets: EC2 CPU Utilization, ALB Request Count, ALB Target Response Time, and Healthy Host Count.
- CPU Utilization Alarm triggers when CPU exceeds 70% on the Auto Scaling Group.
- Unhealthy Host Alarm triggers when any EC2 instance behind the ALB becomes unhealthy.
- Two additional Auto Scaling alarms were automatically created for scale-out and scale-in events.

5. LESSONS LEARNED & FUTURE IMPROVEMENTS

WHAT I LEARNED

- How to design a full-stack application on AWS using multiple services working together cohesively.
- The importance of CloudFront behaviors and origin configurations, especially how mixed content issues arise and how to solve them in the system design.
- How Launch Template user data scripts function and how to fix them when they don’t run as planned.
- How Auto Scaling Groups automatically replace unhealthy instances, creating a system that fixes itself.

- How to use EC2 Instance Connect as a quick, key-free way to troubleshoot and set up instances right from the browser.

WHAT I WOULD DO DIFFERENTLY

- Set up an automated deployment pipeline so updates are pushed to the servers automatically instead of manually
- Add HTTPS directly to the Load Balancer so all traffic is encrypted end-to-end, not just between the user and CloudFront.
- Use a process manager to keep the Node.js application running so it restarts automatically if it crashes, rather than running it in the background..

FUTURE IMPROVEMENTS

- Add a custom domain name instead of the default CloudFront URL.
- Add CloudWatch alarms to send email notifications when alarms trigger.
- Enable logging on the EC2 instances to track errors and monitor activity
- Add an automated pipeline so code updates are automatically pushed to the servers
- Add a web application firewall to CloudFront

APPENDIX

The screenshots are included in the GitHub repository under the /screenshots folder:

DEPLOYED RESOURCES

- EC2 Instances dashboard showing 2 active and healthy instances
- Auto Scaling Group setup and activity history
- DynamoDB tables with preloaded projects and skills data
- Load Balancer target group showing healthy targets
- S3 bucket containing the uploaded index.html file
- CloudFront distribution with origins and behaviors configured
- CloudWatch dashboard with all 4 monitoring widgets
- CloudWatch alarms list

APPLICATION RUNNING

- BGalarza CloudFolio live at CloudFront URL (https://d1w6d84np1ryrj.cloudfront.net)
- Projects section loaded with DynamoDB data
- Skills section loaded with DynamoDB data
- Contact form successfully submits to DynamoDB

GITHUB REPOSITORY

Repository URL: https://github.com/beaglrz/bgalarza-cloudfolio

Includes: README.md, architecture diagram, source code, AWS CLI scripts, and screenshots folder.